

Agent instructions and the cost of working well

Assessment

Needle's central idea remains sound: make intent, ownership, current state and evidence available to colleagues who arrive without the previous session's context. Those concerns have close counterparts in contemporary agent projects. They are not evidence that the system needs rebuilding.

The strongest locally demonstrated defect was the recursive review obligation. A repair could create a new reading obligation, whose findings created more repairs, without a reliable boundary around the work. The former validator also failed to accept a justified rejection of certain allegations. That combination could turn quality assurance into work generation. The September 11 finite-review change addresses that mechanism directly. It does not yet establish how much of total execution cost it caused.¹

The remaining doctrine is **mostly sensible in purpose, but not merely a collaboration and ground-truth agreement in effect**. It also prescribes consolidation, delegation, records, searches, checks, integration and closure. Those actions consume real capacity. Several can expand the task even when the original intent is already satisfied. The next improvement should be selective simplification of those methods, with their useful guarantees preserved.

This assessment recommends keeping the finite workflow, observing its actual use, and then testing a lighter delivery of the remaining instructions. It does not recommend Needle 2, abandoning Kanban, removing all ratchets, or importing another project's workflow wholesale.

Scope and evidence

The public-source snapshot was collected on September 11, 2026. The core sample comprises twelve widely used coding-agent repositories spanning provider tools, independent products and different architectures; one, Roo Code, is now archived and is retained as a labeled historical comparison. Three additional projects broaden the comparison: OpenClaw as a general agent platform, and Superpowers and Spec Kit as workflow systems. This is a purposive sample of popular and relevant projects, not an exhaustive ranking. Star counts measure visibility, not software quality or instruction effectiveness.

Repository instruction files answer **how contributors should work on the agent project**. They do not necessarily expose the agent's own runtime prompts, a user's private global instructions, or the production team's complete operating practice. The comparison follows relevant nested files and workflow skills where available and labels those layers separately. Missing root files are recorded as missing evidence, never as proof of an instruction-free system. The accompanying inventory records default-branch revisions, retrieved paths and source links. Reading focused on the sections relevant to this comparison; long references were not audited exhaustively, and CI enforcement was not independently executed.

Three kinds of evidence have different weight. Public instructions establish a published practice. Vendor engineering accounts establish what their authors report trying. Controlled experiments estimate effects in their tested settings. None directly measures the cost or defect rate of Dennis's current system. The local doctrine was read at the post-#131 revision, a6eb528; the historical loop diagnosis was used as historical evidence rather than treated as today's behavior.¹²

Public repository comparison

Project	Stars at collection	Root instruction words	Inspected layer	Relevant published practice
openclaw/openclaw	389,436	2607	Root + maintainer/autoreview skills	Outcome ownership, proportional proof, finite independent review; renewed review for material changes or concerns. Autoreview default P0 differs from Needle.
obra/superpowers	285,162	1383	Contributor rules + shipped workflow skills	Formal staged work; task review loop capped and scoped; final repairs batched. Contributor rules require real problems and skill-change evaluations.
anomalyco/opencode	206,688	1234	Root + scoped app guides	Local style and dependency direction; actual-implementation tests; tests and typechecks run from package directories.
anthropics/claude-code	144,747	Absent	No root file found; shipped review plugins	Review command uses parallel readers and validation to filter findings. It ends with findings; it does not itself implement a recursive repair workflow.
github/spec-kit	135,595	4037	Root + own constitution + review skill	Substantial integration instructions; binding architecture/test gates and amendment propagation; review expects behavior and regression evidence.
openai/codex	123,356	3128	Contributor rules; root + TUI	Specific-crate tests first; full suite conditional; no repeat after fmt/fix. Integration tests and bounded context are explicit review concerns.
google-gemini/gemini-cli	106,917	616	GEMINI.md + package guides	Heavy preflight at the end; targeted commands while fixing; memory/performance tests conditional. Documentation changes skip local preflight.
earendil-works/pi	104,094	1171	Root development rules	Check dependency types instead of guessing; preserve concurrent work; mandatory check after code, targeted modified tests, no blanket full test run.
OpenHands / OpenHands	87,425	14492	Frontend root + custom review guide	Long architecture/operations reference. Behavioral tests with minimum sufficient cases; one review decision; human owns blocking decisions. Backend is a separate repo.
cline/cline	67,833	723	Root + .clinerules + PR workflow	Environment/build facts and non-obvious lessons. PR-comment workflow proposes apply/skip/respond dispositions, then waits for human approval.
aaif-goose/goose	54,130	865	Root + CLAUDE pointer + docs guide	Issue-owned scope with maintainer exceptions; concise records; conditional local build/test instructions, while clippy remains required before merge.
Aider-AI/aider	48,899	Absent	No root file found; CONTRIBUTING + user docs	Contributor setup/testing guidance; product docs support read-only convention files. No conclusion about a private or runtime harness.
continuede v/continue	35,870	Absent; nested 568	Nested CLI AGENTS.md	CLI commands and architecture; run created/updated tests and formatting/lint before PRs. Root absence does not describe all Continue workflows.
langchain-ai/deepagents	29,323	1340	Root + package/development guides	Observable-behavior tests; rejects implementation-mirroring tests. Optional just-in-time wiki; documented unknowns do not automatically become requirements.

Project	Stars at collection	Root instruction words	Inspected layer	Relevant published practice
RooCodeInc/Roo-Code	24,303	65	Archived repo; root + .roo rules	Tiny root names a concrete state/race trap. Broader rules require tests; PR-fixer mode ends at addressed comments, green tests and no conflicts.

Counts are whitespace-delimited words in the named root, not total loaded guidance. Duplicate CLAUDE aliases are not added to the root count. Roo Code is archived at collection and is a historical comparison, not evidence of an active current team. Goose and Pi links use their current canonical repository names. Full inspected-file links and content hashes are in the [source inventory](#).

What the examples establish

There is no single prevailing amount of process. Some projects put concrete commands and local conventions in short entry files. Others carry substantial architectural rules and elaborate workflows. Length is therefore a useful inventory measure, not a quality score: a small entry file can load a large instruction tree, and a long file can contain valuable project-specific facts.

The closest comparison to Needle is a system that coordinates continued work across sessions, not a repository accepting occasional contributions from human developers. Contributor approval and disclosure rules often protect a public maintainer from unsolicited work. Dennis has already authorized a team to execute inside his projects; copying those external approval steps would add a barrier without preserving the same intent.

Superpowers demonstrates both sides of the issue. Its root contributor guidance demands evidence of a real problem and evaluations for changes to behavior-shaping skills. Its development skill still mandates review, but scopes repair readings, limits task fix rounds to five and specifies a decision when that limit is reached. It discourages rerunning tests already evidenced on the same revision and batches a final repair wave, citing repeated setup and testing as a cost. This is published workflow design, not a measured guarantee that five rounds are optimal.³

OpenClaw provides a closer match to the finite workflow. Its maintainer skill calls for independent review of nontrivial code, resolution of verified findings, and renewed review for substantive changes or unresolved concerns. It explicitly rejects an additional mandatory review pass. However, its separate autoreview skill defaults to a P0-only threshold. That threshold is a material difference from Dennis's instruction to fix all verified in-scope findings; the useful similarity is the stop rule, not a reason to adopt its severity policy.⁴

Spec Kit is a counterexample to the idea that sophisticated projects have abandoned formal rules. Its own constitution binds architecture, behavioral tests, integration parity and security checks, and requires justified complexity and propagated amendments. Its small code-review skill asks for positive and negative behavior tests and before/after regression evidence. These show that detailed constraints remain in active use; they do not establish the payoff of every constraint.⁵

Concrete safeguards that also limit unnecessary work

The Codex contributor guide combines extensive rules with selective checking: package tests first, broader tests for shared-core changes, and no repeat after formatting or lint fixes. Gemini CLI similarly reserves its heaviest preflight for the end and uses targeted checks during repair. These are useful examples of preserving integration confidence while avoiding the same expensive work after every edit.¹⁴

Deep Agents explicitly rejects tests that mirror implementation details and treats generated wiki material as optional context. Cline records non-obvious lessons but excludes easily discoverable facts. Its PR-feedback workflow allows recommending that a comment be skipped. Pi combines source inspection and concurrent-work protection with explicit limits on build/test execution. These examples support ground truth and collaboration while also distinguishing useful verification from routine activity.¹⁵

None is free of tradeoffs. A short Roo root sits alongside broader mode-specific rules; OpenHands carries a much longer root than Needle; and Goose distinguishes local development testing from merge checks. We should compare effective obligations across layers, not reward a short filename in isolation.

Vendor experience is informative, not unanimous

OpenAI's February engineering account describes an agent-built product supported by a repository knowledge base, isolated worktrees, observable running software and mechanical architecture checks. It also reports that a large injected instruction manual failed, replacing it with an approximately 100-line entry map and deeper documentation. Crucially, the same account describes repeated agent review until reviewers are satisfied. It also favors minimal blocking merge gates. This is evidence against claiming that the industry has rejected review loops, and evidence for assessing each mechanism in its operating context.⁶

Anthropic's March account is especially relevant to harness subtraction. Removing many components together initially lost performance and made attribution difficult. The author then removed components individually as model capability improved: context resets and sprint structure became unnecessary, while planning and evaluation retained value in harder cases. Evaluation moved to the end of the build rather than every sprint, but the example still includes multiple build/QA rounds. The lesson is conditional, measured simplification, not a universal one-pass recipe.⁷

Claude Code's current guidance recommends concise broadly applicable instructions, on-demand skills for occasional workflows, and project facts the agent cannot infer. It recommends observing behavior after instruction changes. It also includes a targeted-test example and warns about oversized instruction files. These are practical recommendations, not controlled evidence that a particular file length or disclosure structure is optimal for Needle.⁸

What empirical research can establish

Repository instructions can increase activity without improving completion

Gloaguen and colleagues' June revision evaluates four model/agent pairings on 300 SWE-bench Lite tasks and 138 CTXbench tasks from twelve Python repositories. Context files increased cost by over 20% on average. Developer-written files had a marginal success gain and generated files a marginal loss; neither was statistically significant against no file. Instructions were followed and caused more exploration and testing. The study does not measure long-term coordination, security or owner oversight. Its length/category ablations do not establish a universal ideal file size. The revised paper, rather than the more easily quoted February wording, governs this assessment.⁹

Instructions can also reduce work

Lulla and colleagues examine 124 small pull-request tasks across ten repositories using a Codex configuration. With instruction files, median runtime fell 28.64% and median output tokens fell 16.58%.

However, semantic correctness was outside scope; a manual sanity check of fifty tasks established nontrivial task-related output, not equivalent quality. This is useful counterevidence to “instructions always waste tokens,” but not proof that faster output was equally correct. Output-token reductions are not the same as total cost or subscription allowance savings.¹⁰

A later two-agent study still leaves substantial uncertainty

Khatri's July preprint tests Claude Sonnet 4.6 and Codex with GPT-5.5 on seventeen tasks from three Python repositories, with fifteen tasks shared across agents and 288 valid correctness evaluations. It finds no measurable correctness difference between absent, always-on and selectively retrieved context. The small number of tasks limits power; selective context is larger in two repositories, and injection channels differ between agents. Its narrower process observations are more actionable than a blanket verdict about context. It does not test Needle's long-lived team or September models.¹¹

Better guidance sometimes helps, under specific conditions

Shepard and Albrecht's June probe-and-refine study reports 33.0% mean resolution versus 28.3% with static guidance and 25.5% without guidance, across four trials with Qwen3.5-35B-A3B at a 200-step budget. Improvements chiefly helped the agent reach a patch rather than increase patch precision. Guidance length remains a confound; the positive result is demonstrated on one model, with a constrained context, and a second model did not reproduce the benefit. This is evidence that useful guidance can be learned from failure, not support for continually appending rules to every session.¹²

Harness components can overlap and dilute each other

Lin and colleagues' May AHE preprint varies tools, middleware, memory and system prompts in a controlled harness. On Terminal-Bench 2, memory-, tool- and middleware-only variants each improve on the seed; the evolved prompt alone regresses. The authors describe redundant verification when components overlap and report that the model's prediction of future regressions is weak. This supports inspecting interactions rather than assuming individually reasonable rules add up to a better system. It remains a benchmark research prototype, not a production validation of autonomous rule-making or a result about Needle.¹³

Together, these studies give no credible universal answer of “more harness” or “less harness.” They support asking what information a rule adds, what behavior it changes, what that behavior costs, and what independently judged outcome improves. A task can comply perfectly with its instructions and still be harder to finish because of them.

Needle's remaining rules

The current shared doctrine contains 3,849 whitespace-delimited words across 402 lines; Needle's project file adds 957 words, and the holder register adds 1,106. These are file measurements, not a measured prompt size: what is loaded, repeated, cached or retrieved differs by session. They also exclude skills, reminders and task briefs.²

Rule family	Useful intent	Remaining source of work	Assessment
Intent and decision ownership (§1–2)	Dennis chooses outcomes; agents execute within that authority	Repeated backbriefs or invented decisions can interrupt authorized work	Keep the ownership distinction; routine implementation should remain autonomous
Session economics (§3)	Avoid careless shortcuts and unfinished promises	Says rereading and rerunning checks cost nothing; requires consolidation and broad delegation	Highest-priority wording to revisit: its literal economic premise conflicts with observed finite capacity
Durable records (§4)	Another session can resume accurately	Recording every decision and rejected alternative can create substantial paperwork	Keep consequential decisions and handoff state; scale detail to what a later reader needs
Mechanical protection (§5)	Important failures are caught without memory	A rule may produce a brittle text check, more maintenance and false alarms	Keep behavior and boundary checks; judge each check by the failure it detects
Truthful completeness (§6)	Dennis can trust status	Can be misread as resolving all nearby imperfections	Keep; completeness refers to the agreed promise and truthful remaining gaps
Outcome learning (§7)	Learn whether changes worked	Every bet can become another artifact, watcher or unclosed obligation	Keep meaningful outcome checks; avoid making routine facts into experiments
Ground truth and reuse (§8)	Avoid invented interfaces and duplicate mechanisms	Universal re-verification and proof-of-search can repeat available evidence or expand exploration	Keep direct evidence for consequential claims; make search and rechecking responsive to uncertainty
Method improvement (§9)	Repeated failure changes the approach	Each annoyance can add a rule, mechanism and follow-up	Preserve the feedback; make removal and simplification first-class candidate fixes
Plans and routing (§10)	Work is visible and has an owner	Detailed plan and independent-routing requirements can dominate tiny tasks	Keep a shared corpus; test proportional detail, not removal of ownership
Truthful board (§11)	Oversight matches actual work	Extra state reconciliation can cost effort, but serves a core product promise	Keep; improve automation rather than outsource truth to Dennis
Isolation and integration (§12)	Concurrent agents do not destroy work or land failures	Repeated full suites, duplicated delegated work and coordination startup	Keep isolation and applicable integration checks; inspect repeated execution on unchanged inputs
Finite review (§13)	Independent challenge and verified repairs	Extra review remains possible for a named risk	Keep the restored rule; measure whether completion now converges
Closure (§14)	Work and its evidence survive the session	Archive/citation/branch/record housekeeping can be disproportionate	Keep trustworthy closure; consolidate and automate the paperwork where useful

These categories overlap. It would be misleading to count the sections and say a particular percentage is “good collaboration.” The important distinction is between **what must remain true** and **what an agent must repeatedly do to demonstrate it**.

The economic premise matters beyond review

“Cost nothing” was intended to prevent agents from rationalizing incomplete work. But as an instruction it removes the basis for deciding that a second identical test run, another general search or another document has little value. Even after the review loop is removed, it can still encourage unnecessary work.

The replacement principle worth discussing is: **complete the agreed outcome and verify consequential risks; spend additional work where it can change the result or confidence in it.** That preserves quality without treating every possible check as free. This report proposes that direction; it does not amend the doctrine.

Ground truth needs a stopping condition too

Direct evidence is valuable when a claim will determine an action. Re-reading an unchanged file simply because another agent read it is not automatically the best use of the coordinator. Verification should match the consequence and the reliability of the evidence. A cheap consequential spot-check may be necessary; replaying an entire completed search can defeat the purpose of delegation.

The same applies to uncertainty. A statement such as “this behavior has not been measured” can be the accurate endpoint of a bounded investigation. Treating every expression of uncertainty as an order for further research can turn truthful reporting into indefinite exploration. The present doctrine already permits distinguishing checked facts, recall and inference; that allowance should remain operationally real.

Ratchets are a mixed set, not one treatment

A test proving that the board cannot launch processes protects an architectural boundary. A text match that mistakes a card number for a CSS color produces different value and cost. The local September 11 diagnosis records exactly that false-positive class in #124. This is evidence that friction exists outside review, not that all architectural tests are harmful.¹

Similarly, checking that a review record contains a reader's name and verification evidence helps preserve accountability, but does not prove the reader actually reviewed the code. The holder register itself distinguishes a named mechanism from proof that the whole intent is held. Preserve that distinction when evaluating existing checks; otherwise a record can become the target instead of the behavior it represents.²

The review-loop hypothesis

The evidence supports three separate conclusions:

1. **Confirmed local mechanism:** recursive repair readings and inadequate disposition paths could keep work open and force needless repair activity. This justified fixing the workflow without waiting for a broad benchmark.
2. **Strong but unquantified hypothesis:** that mechanism was a major contributor to recent slowdowns. Long review histories support investigation, but their finding counts also include real defects. No valid cost share has been measured.
3. **Not established:** everything else is efficient, or repeated review generally makes software worse. The local pilot did not complete the comparisons needed to show either. External projects and experiments show benefits and costs depending on scope, task and model.

The earlier broad HR trial and the missing-page review-policy pilot should therefore remain evidence records, not be promoted into a verdict. The September 11 diagnosis records that the broad trial produced no builders and the pilot did not complete the model-policy pairs. This report did not rerun those trials or re-audit their live state.¹

It also cannot establish historical originality. Today's sample shows that repository memory, explicit intent, isolation, scoped autonomy and verification are shared approaches. Determining who first developed a particular combination would require a separate dated historical study and would add little to the immediate decision.

Recommended next step

First, let the finite workflow produce ordinary completed work. Read the next small batch of suitable cards using records already generated. Look for a return of compulsory review recurrence, repeated suite runs on unchanged code, time waiting for colleagues, and record-only repair work. Also check for escaped defects and owner rework. This is operational monitoring, not a causal trial; differences in task difficulty and machine availability must remain visible.

Then test the smallest credible remaining concern: the instruction burden around an otherwise unchanged workflow. Use the existing frozen research assets where they remain suitable. Compare the current post-#131 rules with a short entry layer that preserves decision ownership, boundaries, commands, truthful handoff and finite review, while placing explanations and occasional procedures behind specific pointers. Both conditions should retain the same implementation acceptance and final checks. Changing instruction delivery is a distinct experiment from disabling tests or removing review.

A first experiment should answer one question rather than rebuild the experiment platform. Use tasks with an already working environment and independent acceptance evidence. Keep model, effort, base revision and runner stable within each pair; preserve the first handoff before any repair; record actual usage and infrastructure failures separately. A small pilot can reveal gross overhead or regression, but cannot certify that two workflows have equal quality. If setup stops, fix or bound the setup problem before interpreting model behavior.

If that comparison yields little benefit, the next candidate is duplicated verification and text-based process ratchets, evaluated individually. Keep the checks that defend meaningful behavior. Remove, narrow or replace a check only when its claimed invariant and observed failures justify that change. There is no need to add a mandatory "proof of value" form to every task to implement this recommendation.

Dennis's intended system is still a useful target: one place to see the work, give direction and trust its progress, while capable agents execute remotely. The strongest research-supported change is to preserve those guarantees and reduce obligations that do not contribute evidence or progress. The doctrine should give agents room to make technical judgments and require them to account for outcomes, without making every act of diligence generate another act of diligence.

Sources

1. [Needle review-loop diagnosis](#), historical assessment made September 11 before #131; [finite workflow implementation plan](#) and [verification record](#). ↗↗↗↗
2. Needle at a6eb528: [shared doctrine](#), [holder register](#), [project rules](#), and [founding intent](#). Word counts measured from these local files with whitespace splitting; current links can evolve. ↗↗
3. Superpowers, inspected revision b36e0829c6d0140e93cfef2ca599b1b07d4a7797: [contributor guidance](#) and [development workflow, review and fix stages](#). ↗
4. OpenClaw, inspected revision af73c58582b559187c94f86275402f54e5ccae58: [working agreement](#), [maintainer review policy](#), [autoreview scope](#) and [severity](#). ↗

5. Spec Kit, inspected revision c173bf19a6654e3b05386ec3599349a55282b897: [contributor instructions](#), [project constitution](#), [code-review skill](#). ↗
6. Ryan Lopopolo, OpenAI, [Harness engineering: leveraging Codex in an agent-first world](#), February 11, 2026. Engineering experience report; reviewed September 11. ↗
7. Prithvi Rajasekaran, Anthropic, [Harness design for long-running application development](#), March 24, 2026. Engineering experience report; reviewed September 11. ↗
8. Anthropic, [Best practices for Claude Code](#), current documentation reviewed September 11, 2026. ↗
9. Gloaguen et al., [Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?](#), June 23, 2026 revision, especially §§4–5. February v1 is superseded for this assessment. ↗
10. Lulla et al., [On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents](#), January 28, 2026, especially §§3.1.8–4. [Authors' workshop PDF](#). ↗
11. Prakhar Khatri, [Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories](#), July 28, 2026, especially §§3.3–3.4, 4.1 and 4.5. Preprint; small task sample and injection/corpus confounds. ↗
12. Asa Shepard and Jeannie Albrecht, [Probe-and-Refine Tuning of Repository Guidance for Coding Agents](#), June 19, 2026 revision, especially results and §9 limitations. ↗
13. Lin et al., [Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses](#), May 18, 2026 revision, especially §4.4, component ablations and limitations. Research prototype. ↗
14. [Codex test selection](#); [Gemini CLI validation guidance](#). ↗
15. [Deep Agents testing and context](#); [Cline knowledge selection](#); [Cline comment dispositions](#); [Pi development rules](#). ↗